Spec: Pack Opening × Market Integration (v2)

Status: Proposed — pending approval, then converts to an implementation plan

Date: 2026-07-07

Deciders: CDO, Eng leads (native, d-sports-api, d-sports-backend)

Supersedes: v1 (2026-07-07)

Inputs: CDO proposal (`PACK_OPENING_MARKET_INTEGRATION_PLAN.md`); full audit of `d-sports-engage-native`, `d-sports-api`, `d-sports-backend`, `Pack-opening`, `D-Sports-Local-Docs`; external compliance/RNG research; ADR-0001 (`d-sports-backend/docs/proposals/onchain-gacha-architecture.md`).

---

0. Decisions locked in this revision

#	Decision	Source
D1	Build on `d-sports-api` **(TS)** as system of record. Keep the Rust legacy-compat shim tracking the updated TS surface so cutover doesn't regress.	User
D2	Minting = **Option B (finite mint count, locked copies-per-tier) as an OPT-IN per-pack mode.** Odds-only infinite pool stays the default. **Both** implemented.	User
D3	**Port the** `Pack-opening` **prototype's look** (video → white-flash → 3D orbit → swipeable stack) into `d-sports-engage-native`. It's a visual reference, not reusable code.	User
D4	**Digital Binder is net-new** (schema + API + UI). Known endgame work.	User
D5	**Replace** `Math.random()` **with a CSPRNG weighted draw.** Harvest the design (not code — none exists) from ADR-0001.	User + research
D6	**In-app admin = pack management only** this phase. Rest of `api.d-sports.org/admin` stays web, migrated later. Documented, not built.	User
D7	Payments = DSports Cash + crypto + RevenueCat. **No Stripe.**	User

---

1. Naming (unchanged, for the record)

- `d-sports-api` = TypeScript / Next.js 16 / Prisma 6 / Postgres — **live prod**, and it hosts `/admin` today (route group inside the app).
- `d-sports-backend` = Rust / Axum / Diesel — **fallback in progress** (Strangler-Fig). Pack/marketplace/collectibles handlers are `TODO` stubs; it proxies `/api/*` to `d-sports-api`.
- `d-sports-engage-native` = Expo/RN app. Prod → TS backend; Rust only for local dev parity.

---

2. What already exists (reuse, don't rebuild)

`d-sports-api` already ships most of the mechanics the CDO plan treats as new:

- **Schema:** `Collectible` (rarity, `supply`, `minted`, teamId), `Pack` (price, currency, `quantity`, `sold`, `status` draft/live, `maxReveal`, `animationId`, `contractAddress`), `PackCollectible` (`quantity` **+** `weight` = the odds mechanism), `PackPurchase` (status, paymentMethod, `openedAt`), `MintedNFT`, `UserCollectible`, `Rarity` enum = `COMMON, RARE, EPIC, LEGENDARY, MYTHIC` (**5 tiers** — plan spec'd 4).
- **Odds engine:** `lib/pack-odds.ts` (`computePackOdds`, `getPackOpenability`), `lib/weighted-random.ts` (`weightedRandomSelectMultiple`), tested (`server/__tests__/pack-odds.test.ts`).
- **Server logic:** `server/pack-actions.ts` (989 lines — pack CRUD, odds, openability), `server/minting-actions.ts` (951 lines — sold-out check, weighted draw of `maxReveal` cards, `MintedNFT` + `UserCollectible` creation, `Collectible.minted` increment).
- **Admin CRUD (web):** `app/(dashboard)/admin/packs/`, `components/Admin/tabs/ManagePacksTab.tsx`, `CreateCollectibleTab.tsx`, `pack-management.tsx`, `pack-testing.tsx`. Role-gated by `UserRole` + `canManageTeam`/`canAccessTeam`, enforced **server-side** on every `/api/admin/*` route.
- **GLB pipeline (works):** seed script → Supabase `pack-assets/teams/{id}/animations/` → `GET /api/admin/team-animations` (list) → dropdown select in `ManagePacksTab` → stored as `animationId`/`modelUrl` → rendered by native `WebGlbViewer` (`components/shop/WebGlbViewer.tsx` / `.web.tsx`). **Gap: no *interactive* GLB upload endpoint/UI found in any committed branch** (see §7.3).
- **Shop/checkout (= the plan's "Market"):** `app/(tabs)/shop.tsx`, `PackDetailModal.tsx`, `CartModal.tsx`, checkout via `lib/api/checkout-api.ts` → crypto + DSports Cash. RevenueCat for IAP.
- **Pack opening (2D):** native `components/wallet/PackOpeningModal.tsx`, `components/shop/PackOpeningView.tsx` — `Animated` + hand-rolled confetti; rarity colors in `lib/rarity-utils.ts` (all 5 tiers). No video, no white-flash, no 3D orbit yet.
- **Compliance groundwork:** `d-sports-engage-native/docs/compliance/odds-disclosure-copy-guidelines.md`, `pack-odds-client-integration.md`.

Consequence: this is ~60-70% an extension of `d-sports-api`. Genuinely new: Binder, in-app admin, interactive GLB upload, the richer opening animation, and the Option-B finite-mint mode.

---

3. Corrections to the CDO plan that still stand

- `PackOpening.tsx` **/** `getRarityColors()` **don't exist.** `Pack-opening/` is a ~550-line vanilla JS/CSS prototype (raw `requestAnimationFrame` orbit, Web Animations API particles, 2 hardcoded rarities). Treat as **visual spec**; the port target is the native `Animated` components, not this code (React Native has no DOM to port the prototype's `querySelector` logic into).
- **Binder is not an existing feature.** Only a static design HTML exists (`D-Sports-Local-Docs/TDD/_binder-preview/TDD-Binder-Preview.html`). Nearest live analog (Wallet, grouped by rarity) is not a binder.
- **"Market" is the existing Shop.** Repositioning is product/UX, not new plumbing.
- **White-flash timing is per-video, not a constant.** Prototype's `WHITEOUT = 7.733s` / `DUR = 8.367s` are tuned to one asset. Must become per-pack metadata (§4.1).
- `POST /admin/packs` **returning** `mint_instances` only makes sense under Option B (§4.2).

---

4. Target design

4.1 Schema changes (`d-sports-api`, additive)

Pack — add:

```
// on model Pack
packSize        PackSize   @default(THREE)   // replaces implicit maxReveal int with explicit enum for the animation layer
videoUrl        String?
```

```
videoDurationMs Int?
whiteFlashMs    Int?        // per-pack, extracted at video upload; NOT a global constant
modelUrl        String?     // GLB (already effectively stored via animationId; formalize)
mintMode        MintMode   @default(ODDS_ONLY)   // D2: opt-in finite mode
totalMintCount  Int?        // Option B only: locked target supply for the pack
enum PackSize { THREE FIVE SEVEN }
enum MintMode { ODDS_ONLY FINITE }
```

Option B finite-mint (D2) — only materialized when `mintMode = FINITE`:

```
model MintInstance {
  id             String    @id @default(cuid())
  packId         String
  collectibleId  String
  serialNo       Int        // 1..copies for this collectible in this pack
  status         String     @default("available")  // available | drawn
  drawnByPurchaseId String?
  pack           Pack        @relation(fields: [packId], references: [id], onDelete: Cascade)
  collectible    Collectible @relation(fields: [collectibleId], references: [id])
  @@unique([packId, collectibleId, serialNo])
  @@index([packId, status])
}
```

- `ODDS_ONLY` (default): current behavior — `PackCollectible.weight` only, nothing decrements, no sold-out per card.
- `FINITE`: on publish, generate `MintInstance` rows per the admin's copies-per-tier allocation. A pull marks an instance `drawn` inside the open transaction; pack becomes unopenable when exhausted. Reconciles with the existing unused `Collectible.supply`/`.minted` fields.
- **Validation (plan §3):** target count must divide across the pool without remainder → surfaced in admin UI as the "adjust to 300 or 304" warning.

Binder (D4):

```
model Binder {
  id         String       @id @default(cuid())
  userId     String
  name       String
  pinHash    String?       // null = no PIN
  coverImage String?
  createdAt  DateTime      @default(now())
  updatedAt  DateTime      @updatedAt
  user       User          @relation(fields: [userId], references: [id], onDelete: Cascade)
  cards      BinderCard[]
  @@index([userId])
}
model BinderCard {
  id            String      @id @default(cuid())
  binderId      String
  collectibleId String
  quantity      Int         @default(1)   // quantity bucket, NOT one row per copy
  shelf         String?                    // league/team key, or null -> "General"
  addedAt       DateTime    @default(now())
  binder        Binder      @relation(fields: [binderId], references: [id], onDelete: Cascade)
  collectible   Collectible @relation(fields: [collectibleId], references: [id])
  @@unique([binderId, collectibleId])
  @@index([binderId])
}
```

- **Quantity-bucket model (revised after plan review):** `UserCollectible` is unique per `(userId, collectibleId)` with a `quantity` counter the mint flow increments — so `BinderCard` cannot be one-row-per-copy. It's a bucket keyed `(binderId, collectibleId)`. Invariant (app-enforced in `binder-actions.ts`): `sum(BinderCard.quantity across a user's binders, per collectible) <= UserCollectible.quantity`. "Loose"/unplaced copies = owned − placed.
- "Open Later" = purchase completes with a chosen `PackPurchase.targetBinderId` (new nullable field) but `openedAt = null`; opening later lands into that binder.
- Transfer = quantity move between two `(binderId, collectibleId)` buckets in one transaction.
- PIN: hashed with salted **PBKDF2** (`lib/binder-pin.ts` — reuses the `pbkdf2` primitive already imported in `server/wallet-actions.ts`, but salted; the wallet code's own PIN hashing is unsalted SHA-256, which we deliberately do NOT copy). PIN gating is UX-level; ownership is the real boundary.

4.2 API (extend, don't replace)

```
# Admin (native + web share these)
POST /api/admin/packs                        exists — extend body: packSize, mintMode, totalMintCount, tier allocation
POST /api/admin/packs/{id}/upload-video      NEW — validates metadata, mints a Supabase SIGNED UPLOAD URL; client uploads direct to s
POST /api/admin/packs/{id}/upload-model      NEW — same signed-URL pattern for GLB (model/gltf-binary + magic-byte check)
                                             (signed URLs because Vercel caps route bodies at ~4.5MB — files must not pass through Ne
GET  /api/admin/packs                        exists

# User
GET  /api/packs/{id}/odds                    EXISTS (app/api/packs/[id]/odds/route.ts) — feeds pre-purchase disclosure AND the public
POST /api/packs/{id}/open                    EXTEND — accept binderId; land pulled cards into that binder atomically; FINITE mode ma
GET  /api/binders                            NEW
POST /api/binders                            NEW (name, optional PIN)
POST /api/binders/{id}/verify-pin            NEW — returns {granted: boolean}; 5-failure/30s lockout. No session token
                                             (ownership is the security boundary; PIN is UX-level)
GET  /api/binders/{id}                       NEW (contents grouped by shelf)
POST /api/binders/{id}/transfer-card         NEW (from/to binder; restrictions per §6)
```

All return the existing `ActionResult<T>` envelope.

4.3 In-app admin (D6) — client-only lift

Admin auth is Clerk-session + server-side role checks, **not web-coupled** — so a native admin surface reuses the same `/api/admin/*` endpoints with no new backend auth:

- New `app/(admin)/` route group in native, gated by a ported `useAdminRole()` (mirror web's `AdminProvider`/`useAdminContext`).
- Screens this phase: pack create/edit, card-pool multi-select, mint/odds config (incl. Option-B allocation UI), pack list. Rebuild `ManagePacksTab`/`CreateCollectibleTab` logic as native screens (not DOM ports).
- **Out of scope this phase** (stays on web, tracked for later migration): user management, revenue/FX, team-view, reports.

4.4 Pack opening animation (D3) — native

Extend the native `Animated` components; use the prototype only as the visual target:

- Video via `expo-video`, driven by per-pack `videoDurationMs`/`whiteFlashMs` (not constants).
- Orbit/stack/swipe: extend existing gesture handling; scale ring by `packSize` (3/5/7).
- Rarity FX: `lib/rarity-utils.ts` already maps all 5 tiers' colors — add per-tier particle/beam/haptic intensity.
- 3D: reuse `WebGlbViewer` for **post-open card detail only** (matches the plan's own assumption).

---

5. RNG & fairness (D5)

Now (this feature, TS): replace `Math.random()` in `lib/weighted-random.ts` with a **Node** `crypto` **CSPRNG** over the weighted cumulative distribution. (`components/Admin/pack-testing.tsx:224` deliberately stays on `Math.random` — it's a client-side simulation that awards nothing and can't import `node:crypto`; it's also being promoted to a **public "preview this pack" feature**: shoppers can run a clearly-watermarked simulated opening drawn from the real disclosed odds before buying — an interactive odds-disclosure asset. The native shop's `PackDetailModal` gets the same preview and drops its hardcoded drop rates for `GET /api/packs/{id}/odds`.) Add an **append-only draw audit log**: per open — userId, packPurchaseId, packId, `PackCollectible` weights snapshot + a hash of the odds table version, RNG output, awarded cards, timestamp. This is the pragmatic, store-defensible default (research verdict below). Low effort, high compliance value.

Later (already a proposed ADR — do not redesign):
`d-sports-backend/docs/proposals/onchain-gacha-architecture.md` (**ADR-0001, status Proposed**) specifies the on-chain-fair path: a **drand quicknet BLS beacon as VRF + commit-reveal**, with the drawing-pool spec **SHA-256 hashed and anchored on-chain**, and **pull-to-mint with finite supply caps** — which aligns with the Option-B model in D2. No Rust code exists yet (no `rand` crate; `hmac`/`sha2` are only used by the oracle webhook verifier). Treat ADR-0001 as the future track;

the CSPRNG+audit-log work above is forward-compatible with it (same odds-table-hash concept).

Research verdict (2026): App/Play stores require pre-purchase *odds disclosure*, not a specific RNG architecture. Server-side CSPRNG weighted draw + versioned odds table + append-only log is the recommended default; commit-reveal is optional (trust-as-feature); on-chain VRF is overkill until the economy is on-chain-first. Sources: Apple loot-box odds disclosure, Google Play requirement, 2026 jurisdiction map.

Also required: verify `GET /packs/{id}/odds` output reaches the **purchase** UI pre-checkout (not just the reveal). PEGI moves to mandatory 16+ for paid random items from June 2026; Belgium bans paid loot boxes; SK/China mandate public probability pages — relevant if those markets are in scope.

---

6. Open questions needing your call

#	Question	My recommendation
Q1	**Transfer restrictions** (cooldown / tier lock / rate limit on `transfer-card`)?	Start with none; add a per-user rate limit only if abuse appears. Confirm.
Q2	**Interactive GLB upload** — is it truly not built (point me at the branch/repo if it is), or do we build the `upload-model` endpoint fresh?	Build the thin endpoint on the existing pipeline; ~half a day.
Q3	**Option-B guarantees** — do FINITE packs still honor tier "guarantees" (e.g. ≥1 Rare, per `getDefaultGuaranteeForTier`) when supply of that tier runs low?	Yes, but block publish if guarantee is mathematically unsatisfiable at the chosen allocation.
Q4	**drand/on-chain fairness (ADR-0001)** — in scope for this feature, or explicitly deferred to the on-chain-gacha track?	Defer; ship CSPRNG+audit-log now (forward-compatible).

---

7. Phased plan (converts to tickets on approval)

Phase 0 — Foundations & compliance

- CSPRNG swap + draw audit log (§5); confirm pre-purchase odds disclosure wiring; resolve Q1-Q4.
- Schema migration: Pack fields, `MintMode`/`PackSize`, `MintInstance`, `Binder`/`BinderCard`.

Phase 1 — Admin pack authoring (native, D6)

- Native `app/(admin)/` route group + role gate.
- Pack create/edit, card-pool multi-select, odds config, **Option-B allocation UI + divisibility validation**.
- `upload-video` endpoint (+ duration/codec validation); `upload-model` **interactive GLB endpoint** (Q2).

Phase 2 — Digital Binder (D4, biggest phase)

- `Binder`/`BinderCard` API (list/create/verify-pin/contents/transfer).
- Native Binder screens: cover carousel, PIN entry, shelf-grouped interior.

Phase 3 — Purchase → open → land, new animation (D3)

- Binder-selection modal at checkout (open now / open later / create-inline).
- Port the prototype look into native opening: video + per-pack timing, 5-tier FX, haptics, packSize-scaled orbit.
- `POST /packs/{id}/open` extended for `binderId` + FINITE-mode instance marking, atomic.

Phase 4 — Transfers, polish, Rust parity

- Card transfer between binders (Q1 restrictions).

- Post-open binder redirect + new-card highlight/badge.
- **Rust parity backlog (D1):** file every new model/endpoint into `d-sports-backend/docs/parity/plans/PARITY_GAPS.md`; ensure the legacy-compat shim (`crates/engage/src/legacy_compat/handlers/packs.rs`, `commerce.rs`) proxies the new `/api/*` routes so native never breaks at cutover. Port schema 1:1 from §4.1 when `crates/collectibles`/`marketplace` leave stub status.

---

8. Carried-over plan questions (resolved)

Plan open question	Resolution
Rarity global vs per-pack	**Global** — matches existing `Collectible.rarity`; per-pack would be a much larger schema change.
White-flash timing	**Per-pack metadata** (`whiteFlashMs`), not a constant.
3D interaction during reveal	**Post-open detail only** — matches existing `WebGlbViewer` usage.
Draft vs live	**Already exists** — `Pack.status` defaults to `draft`. Not new work.
Binder creation in purchase modal	**Yes** — cheap once Binder API exists.
Minting model	**Resolved: D2** — Option B opt-in, odds-only default.